

APACHE SPARK ARCHITECTURE → UNDERSTAND HOW SPARK PROCESSES BIG DATA AT SCALE



BY MICHAEL FERNANDES

WHAT IS APACHE SPARK

Apache Spark is an open-source distributed computing framework designed for processing massive datasets across clusters of machines.

Why Spark Became Industry Standard

Up to 100x faster than traditional disk-based systems

Unified engine for Batch, Streaming, SQL, ML & Graph Processing

Scales from a single machine to thousands of nodes

Fault-tolerant and highly distributed

Apache spark is used by

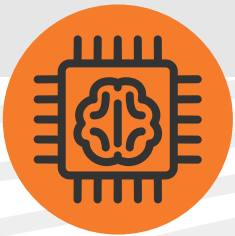


WHY APACHE SPARK?

Imagine processing 100 TB of data on a single machine. The Challenges faced are

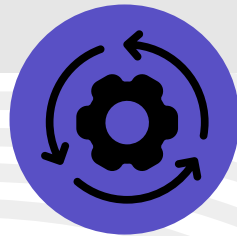
Limited Memory

A single machine can only store and process a limited amount of data in RAM.



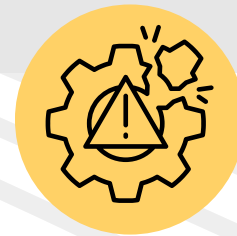
Slow Processing

Processing terabytes of data sequentially can take hours or even days.



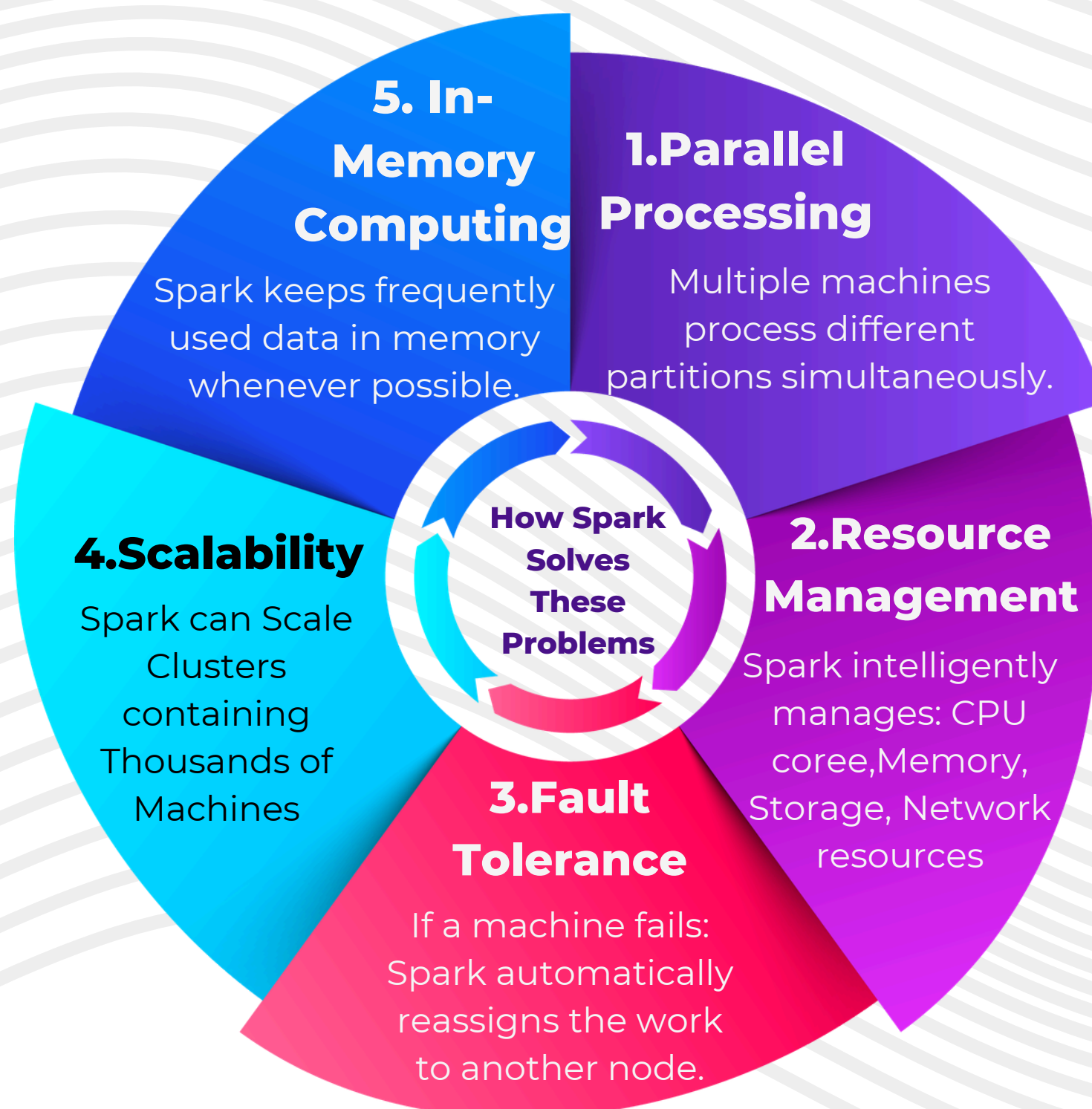
Single Point of Failure

If the machine crashes, the entire processing job fails.

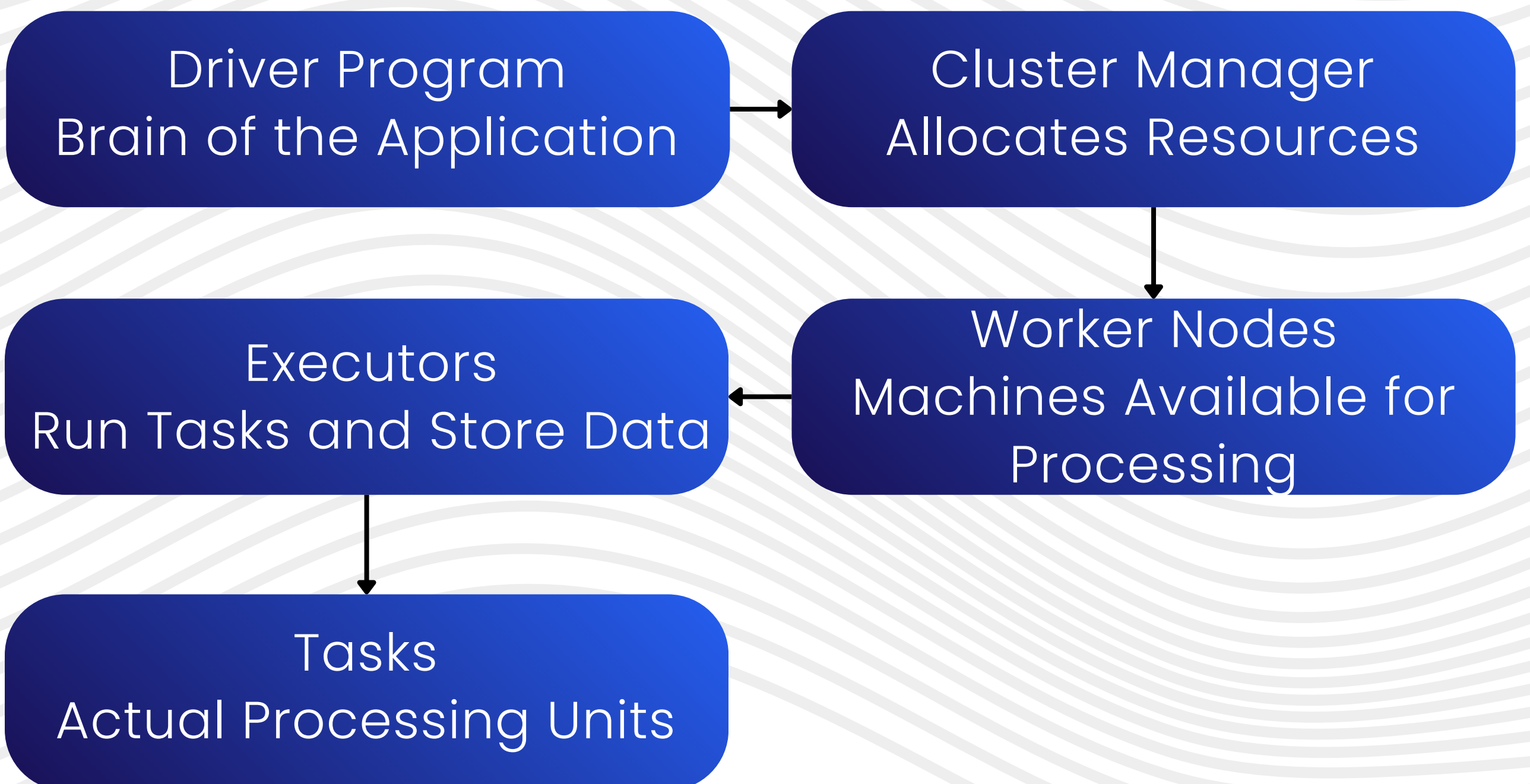
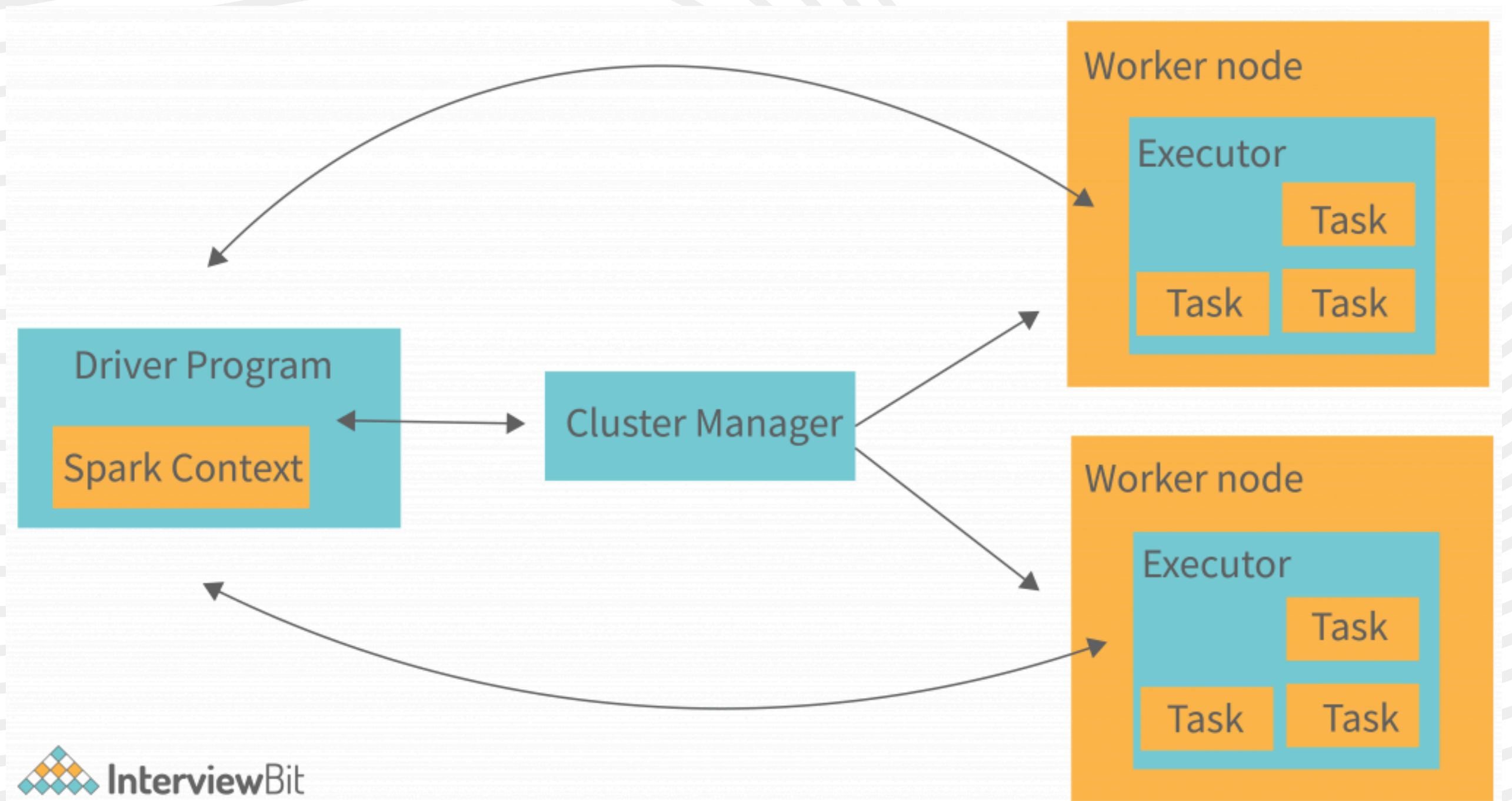


Limited Scalability

Adding more data eventually exceeds the capabilities of a single server.



HIGH-LEVEL SPARK ARCHITECTURE



DRIVER PROGRAM (THE BRAIN)

Creates SparkSession

Entry point of Spark application.

Builds DAG

Converts code into execution plans.

Schedule Jobs

Determines how work should be executed.

Coordinates Executors

Sends tasks to worker nodes.

Collects Results

Receives processed results from executors.



```
spark.read.csv()  
.filter()  
.groupBy()  
.count()  
.show()
```

The Driver interprets this code and determines the most efficient execution strategy.
Without the Driver, Spark cannot coordinate distributed processing.

SPARK SESSION

SparkSession is the entry point to every Apache Spark application. It provides a single, unified interface to interact with all Spark functionalities, including DataFrames, SQL, Streaming, Machine Learning, and Graph Processing.

1. Creates the Spark Application

SparkSession initializes the Spark application and establishes communication with the Spark cluster.

2. Reads Data from Multiple Sources

SparkSession can load data from various formats, such as CSV, JSON, Parquet, Delta Lake

3. Creates and Manages DataFrames

SparkSession enables developers to create DataFrames, Spark's primary abstraction for structured data.

4. Executes Spark SQL Queries

It allows SQL queries to run directly on DataFrames.

5. Configures Spark Applications

SparkSession manages application-level configurations such as:

- Memory allocation
- Executor settings
- Shuffle partitions
- Time zone
- Serialization

6. Integrates with the Spark Ecosystem

SparkSession provides access to nearly every Spark component:

- Spark SQL
- MLlib
- GraphX (Scala)
- Delta Lake
- Hive

CLUSTER MANAGER

Resource Allocation Layer

A Spark application cannot run unless resources are allocated.

This is the responsibility of the Cluster Manager.

Standalone

Built into Spark

YARN

Most common in Hadoop ecosystems

Kubernetes

Cloud-native deployments

Mesos

General-purpose cluster manager

CPU Allocation

Memory Allocation

Executor Creation

Cluster Monitoring

The Cluster Manager determines:

"Which machines will run this application?"

WORKER NODES & EXECUTORS

Worker Node

A machine that contributes CPU, memory, and storage resources.

A cluster may contain:

- 10 Nodes
- 100 Nodes
- 1000+ Nodes

Executor

A JVM process launched on Worker Nodes.

Executor Responsibilities

Execute Tasks

Processes assigned data partitions.

Cache Data

Stores data in memory.

Shuffle Data

Transfers data between stages.

Return Results

Sends outputs back to Driver.

JOBS, STAGES & TASKS

1. What is a Job?

A Job is the highest unit of execution in Spark.
A new Job is created only when an Action is called.
Until then, Spark simply records the transformations (thanks to lazy evaluation).
Although multiple transformations were written, Spark creates one Job only when `show()` is executed.

2. What is a Stage?

A Stage is a logical group of tasks that can execute without requiring data to be exchanged between executors.
Spark divides a Job into multiple Stages based on Shuffle Boundaries.

Two Types of Stages

ShuffleMap Stage

- Performs transformations before a shuffle.
- Produces intermediate data.

Result Stage

- Final stage of the Job.
- Produces the output for the requested Action.

JOBS, STAGES & TASKS

3. What is a Task?

A Task is the smallest unit of execution in Spark.
Each Task processes one data partition.

Tasks are sent by the Driver to Executors, where they run in parallel.

Example

Dataset Size: 500 GB

Partitions: 100

Spark creates:

- 100 Tasks

Each Task processes one partition independently.

This parallel execution is one of the main reasons Spark is so fast.

Key Takeaway

- Job → Created when an Action is executed.
- Stage → A group of Tasks separated by shuffle boundaries.
- Task → The smallest execution unit, processing one partition.

END-TO-END ARCHITECTURE

The process begins when you run a Spark application using `spark-submit` or execute code in a PySpark notebook.



01 USER SUBMITS SPARK APPLICATION

SPARK SESSION IS CREATED 02



Spark initializes the `SparkSession`, which serves as the entry point to the Spark application.

- Initializes the application
- Creates or retrieves the `SparkContext`

The Driver Program becomes the central coordinator. Its responsibilities include:

- Reading your application code
- Creating the logical execution plan



03 DRIVER PROGRAM STARTS

CLUSTER MANAGER ALLOCATES RESOURCES 04



The Driver requests computing resources from the Cluster Manager.

The Cluster Manager Finds available Worker Nodes and Allocates CPU cores and memory

Executors are JVM processes started on Worker Nodes. Each Executor:

- Receives tasks from the Driver
- Processes assigned data partitions
- Stores cached data in memory



05 EXECUTORS ARE LAUNCHED

TASK SCHEDULER CREATES TASKS 06



Each Stage is divided into Tasks.

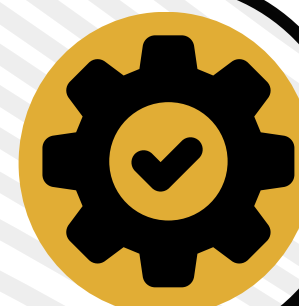
A Task processes exactly one data partition.

The Task Scheduler assigns these Tasks to available Executors for parallel execution.

Executors execute Tasks simultaneously across multiple Worker Nodes.

During execution they:

- Read partitioned data
- Perform transformations



07 EXECUTORS PROCESS DATA IN PARALLEL

RESULTS ARE RETURNED AND STORED 08



Once all Tasks are complete:

- Executors send results back to the Driver (for actions like `show()` or `collect()`).
- Or write the processed data directly to storage systems

The background of the image consists of numerous thin, light gray lines that flow horizontally across the frame. These lines are not perfectly straight but have a gentle, undulating wave pattern, creating a sense of movement and depth. The lines are more densely packed in some areas and more spread out in others, contributing to a textured, organic feel.

THANK YOU